Confidently Wrong: Ranking Inversion in Cross-Network Denial-of-Service Detection

Rajveer Singh Pall

Abstract

Flow-based DoS detectors report accuracies above 99% and lose most of that skill on a different network. The loss is well documented; its character is not. Where the score ordering survives the move, the repair is a new threshold; where it has broken, no threshold is enough and the scoring function itself has to change.

We train four model families on four independently captured testbeds under a published common feature set and evaluate every ordered pair over five seeds. Transfer does not decay toward chance; it passes through chance. Of 240 run-level evaluations, 108 score below ROC-AUC 0.5, three of the twelve ordered pairs do so in all four model families, and the worst reaches 0.0697 — negating the score would give 0.9303. Attack flows are being scored below benign ones.

Most pairs fail at the ranking rather than at the threshold, nine to eleven of twelve for the primary model depending on how the two losses are compared, which makes recalibration the wrong instrument for most of them: a threshold can only cut an ordering it cannot repair. Ten deliberately balanced labelled target flows lift median macro-F1 from 0.3919 to 0.7253 when added to the source data, but a classifier trained on those ten flows alone does at least as well (0.7852): here the labels, not the source model, do the repair. Cross-network evaluation has to separate ranking failure from threshold failure, because the two admit different fixes.

Keywords: network intrusion detection, DDoS, cross-dataset generalisation, distribution shift, calibration, evaluation methodology

1 Introduction

Published flow-based DDoS detectors report accuracies above 99%. Deployed ones do not behave that way. The gap between those two facts has been measured repeatedly: models trained on one intrusion-detection dataset and tested on another lose most of their apparent skill [1, 2]. That finding is established.

What the gap *means* is not, and the ambiguity is expensive: an operator who sees performance fall after a move to a new network has to choose between recalibrating the threshold and replacing the detector, and two readings of the same collapsed score prescribe opposite choices. The familiar reading is quantitative. The model is somewhat overfitted to its training network and its performance degrades, so the remedies are the standard three: engineer features that describe the attack rather than the capture, monitor the distance between the training and deployment distributions, and recalibrate the decision threshold on arrival. The second reading is that the detector has not lost its signal but reversed it. If that is what happens, recalibration cannot help at all, because it moves a cut point along an ordering that is already wrong, and the forms of feature engineering and drift monitoring evaluated here do not address it either. All three assume the model still orders traffic in a useful direction.

Separating the two readings takes one change of instrument: report ROC-AUC for each ordered source-target pair instead of accuracy in aggregate. A detector that has lost its signal sits at 0.5. A detector that has reversed it sits below 0.5, and the further below, the more information it carries with the wrong sign. We call the first a **threshold failure**, which moving the decision boundary can repair, and the second a **ranking failure**, which only a change to the scoring function can repair. Where a ranking failure goes past chance and the ordering is not merely lost but systematically reversed, we call it **ranking inversion**. Which of the two a deployment suffers is the question this paper answers, and the answer decides what an operator should do about it.

Running every ordered pair among four independently captured testbeds, on the NetFlow v2 common feature set, with exact de-duplication and source-only preprocessing, we find that transfer does not decay toward chance. It passes through chance. Mean external ROC-AUC is 0.5054; 108 of 240 run-level external evaluations fall *below* 0.5, as do 5 of the 12 ordered pairs once seeds are averaged; one pair reaches **0.0697**.

That last number is what an aggregate near chance conceals. An AUC of 0.0697 is not weak evidence; it is strong evidence with the wrong sign. Attack flows are systematically assigned lower scores than benign flows, and negating the detector's output would yield an ROC-AUC of approximately 0.9303. This is a statement about ranking, not about accuracy: ROC-AUC is threshold-free, and the accuracy of a negated detector would still depend on the operating point and the target prevalence. A model at chance carries no usable ordering. A model at 0.0697 carries a strong ordering with the sign reversed.

Monitors that summarise the input distribution as a whole do not see this coming (§4.10). Drift monitoring watches $P(X)$, the distribution of the features. What decides whether a detector still works is $P(Y \mid X)$, the relationship between features and labels. The two need not move together: a target network can look statistically familiar and score differently, or look alien and score the same. That is why the divergence measures in §4.10 fail as risk predictors while a measure of the model's own behaviour does not.

A plausible mechanism is easy to state. If high throughput marks attack traffic in one testbed and marks the busiest legitimate hosts in another, a decision surface fitted to the first is close to optimal-but-negated on the second. Two of our own results constrain what that mechanism can be. Logistic regression inverts on the same pairs as the tree ensembles (§4.3), so it cannot require high-order feature interactions, which a linear model cannot represent. And feature count is uncorrelated with external ranking quality (§4.7), so it is not a matter of model capacity either. The reading consistent with both constraints is that individual feature-label associations change between these networks. Consistent with is not established by: we do not isolate which associations or how they change, and §6 says so.

1.1 Contributions

1. **The principal finding: cross-network failure can invert the ranking rather than attenuate it.** Three of twelve ordered pairs score below chance in all four model families, logistic regression included, and the worst reaches ROC-AUC 0.0697.
2. A failure taxonomy separating threshold failure from ranking failure, decomposed for every ordered pair, with evidence consistent with a testbed-lineage effect on which mode appears (§4.14).
3. A pre-specified capacity-feature hypothesis that the data does not support, alongside a controlled random-subset experiment showing that feature count acts on the operating point rather than on the external ranking.
4. A label-free triage analysis in which Average Thresholded Confidence beats the mean-gap baseline under leave-one-pair-out evaluation while three marginal distribution-distance measures do not.
5. A benchmark-integrity audit covering exact duplication, contradictory labels under the full attack taxonomy, non-finite extractor artefacts and a regenerated release of CICIDS2017, showing that de-

duplication and the choice of release materially change measured transfer performance while contradictory-label exclusion does not, under the primary DoS-only setting.

Section 2 places the study against prior cross-dataset work and against the contaminant-feature literature. Section 3 gives the data and the protocol, including the de-duplication and threshold decisions the results turn out to depend on. Section 4 reports the transfer matrix, applies the taxonomy, tests what causes the inversion and measures what repairs it. Sections 5 and 6 give the practical reading and the limits.

2 Related work

Four strands bear on this study: the measured size of the cross-dataset gap, the feature spaces those measurements are taken in, the quality of the underlying captures, and what can be estimated about a deployment before any target label exists. Each strand has established that something goes wrong. None of them settles what kind of failure it is.

2.1 The cross-dataset gap

That flow-based intrusion detectors transfer badly is not in dispute. D'Hooge et al. [1] established reference scores on CICIDS2017 and CSE-CIC-IDS2018 across twelve supervised algorithms and concluded that the classifiers could not operate reliably on traffic from a network unseen at training time. Cantone et al. [2] extended the design to four datasets and four classifiers, reported cross-dataset accuracy largely commensurate with random chance, and traced part of it to anomalies in the data itself. The question has stayed open: Hossain et al. [3] report cross-dataset accuracy below 40% between UNSW-NB15 and ToN-IoT for models scoring above 95% within either one, and Elangovan et al. [4] find the same pattern across four datasets when generalisation is tested temporally and semantically rather than within a single capture. Apruzzese et al. [23] formalised cross-evaluation of NIDS across labelled datasets as a protocol in its own right, and Verkerken et al. [25] report that unsupervised detectors generalise across CIC captures better than supervised ones.

The closest prior study ran our primary design. Layeghy and Portmann [22] trained tree ensembles, feed-forward and recurrent networks and an isolation forest on the same four NetFlow v2 datasets, in every ordered pair, found that no model generalised across all of them and that transfer was strongly asymmetric, and used SHAP to attribute the failure to feature–label associations that hold in one dataset and not in another. Our protocol is theirs in its essentials, and their explanation is consistent with everything we report; we do not offer a competing mechanism. What we change is the instrument. They report F1; we report ROC-AUC per ordered pair, which separates a detector whose ranking has reversed from one whose ranking survives under a misplaced threshold. The two failures call for different repairs, which is why we test a representative version of each rather than proposing a new one.

2.2 Shared feature spaces, and what hides inside them

Sarhan et al. [5] identified the absence of a common feature set as a structural obstacle to cross-dataset claims, and released NetFlow v2 conversions of four datasets sharing 43 fields, and compared NetFlow and CICFlowMeter feature sets for how well models built on them generalise [24]. We adopt that feature set precisely so our transfer results cannot be attributed to ad-hoc harmonisation, and verify the shared space by assertion rather than assumption. NetFlow versions that add temporal features have since been released [30]; we use v2, the version with the largest body of prior cross-dataset results, and leave replication on the newer versions open. The four sources are CSE-CIC-IDS2018 [6], BoT-IoT [7], ToN-IoT [8] and UNSW-NB15 [9]; the secondary study uses CIC-DDoS2019 [10] and CICIDS2017 [6].

A standard feature space settles one problem and exposes another. A second line of work asks not whether the labels are right but whether the features are honest. D'hooge et al. [11] showed that metadata fields, destination port above all, act as shortcuts strong enough to separate ten intrusion-detection datasets at 70–100% accuracy on their own; D'hooge et al. [12] extended the search to non-metadata contaminants; and D'hooge et al. [13] showed how much of the apparent value of a large feature set is carried by a small block of it. Goldschmidt and Chudá [14] survey 89 public NIDS datasets and reach a compatible conclusion about how dataset construction shapes what models learn.

Our `DNS_QUERY_ID` result (§4.7) is an instance of the phenomenon that literature describes, and we do not present shortcut features as a discovery. What is new here is what a shortcut does across networks rather than within one. The same feature space that inflates in-domain scores can also produce a reversed target ordering, and which of those two happens decides whether recalibration or target supervision is the right response.

2.3 Dataset quality and label error

Engelen et al. [15] documented labelling and traffic-generation defects in CICIDS2017 and released a corrected version; Lanvin et al. [16] extended the analysis and quantified the effect of the corrections on measured performance. Liu et al. [21] then documented labelling, feature-generation and attack-orchestration errors in both CICIDS2017 and CSE-CIC-IDS2018 and released regenerated flows for both; the corrected CICIDS2017 used in this paper is that 2022 release. These studies examine in-domain consequences. We use the regenerated release differently, as an instrument: one identical source model is applied to the original and the regenerated target. Because the regeneration changes flow features as well as labels, the comparison measures how much a cross-dataset result depends on which release is used rather than isolating label error, and the result runs opposite to the intuitive direction (§4.15).

Flood et al. [28] catalogue design flaws across seven benchmark NIDS datasets that bias experiments built on them, and Layeghy et al. [29] show that the benign traffic of synthetic benchmarks differs statistically from real network traffic. Our duplicate accounting (§3.3) is adjacent to this work but, to our knowledge, has not been reported at this scale for the NetFlow v2 datasets. It is measured on the publicly mirrored copies described in §3.1, not on the original releases. That three of four are more than 80% duplicate in the model's own feature space constrains how in-domain numbers computed on them should be read, and §4.9 measures how much it moves the transfer results themselves.

2.4 Evaluation norms, and what is knowable without target labels

Arp et al. [17] catalogue ten recurring pitfalls in machine learning for security, of which sampling bias, data snooping, spurious correlations and the base-rate fallacy bear directly on cross-dataset evaluation. We map the protocol against all ten in Table 2 rather than asserting compliance in prose. Axelsson [18] established that intrusion detection is dominated by the base rate; the datasets here span base rates from 1.3% to 77.4%, so balanced-test-set F1 is not comparable across them and operating-point metrics are reported alongside.

An operator meeting a new network has traffic but no labels, which makes estimating target performance from unlabelled data the practical question. Garg et al. [19] introduced Average Thresholded Confidence, which learns a confidence threshold on source data and predicts accuracy as the fraction of unlabelled target points above it. The competing family measures the distance between the source and target feature distributions directly, which is also the theoretical motivation for domain-adversarial training. Section 4.10 evaluates both families on the same twelve ordered pairs and finds their ordering reversed relative to the assumption embedded in most of that work: three aggregate distribution-distance measures lose to a mean baseline, and the confidence-based estimator does not.

Adapting a detector to a new network is a well-populated area in its own right, including covariance

alignment [20] and domain-invariant representation learning for NIDS specifically: DI-NIDS [26] learns adversarially domain-invariant features across NetFlow datasets and reports that earlier domain adaptation for NIDS had limited success, and Singla et al. [27] use adversarial adaptation to prepare a detector for a network with little labelled data. We make no methodological contribution there and do not claim one. We evaluate only one shallow adaptation baseline; CORAL appears in §4.11 as the standard zero-label baseline, and the contribution is the measurement: what the available methods that use no attack labels recover in this setting, which is little and uneven, against what a handful of labelled target flows recovers, which is most of the gap.

3 Data and experimental protocol

The design in one paragraph. Four intrusion-detection datasets, each captured on a different testbed, are reduced to a single shared 39-feature space. For each dataset in turn, four model families are trained on 80% of it and evaluated twice: on its own held-out 20%, which we call **in-domain**, and on each of the other three datasets, which we call **external**. In the primary transfer experiment nothing about the target is used in fitting: not its labels, not its feature statistics, not its threshold. The quantity of interest is the difference between those two evaluations, and every design decision below exists to ensure that difference measures domain change rather than an artefact of preparation.

3.1 A published feature space, verified

Cross-dataset claims in this area usually rest on intersecting the column names of two CICFlowMeter exports. The resulting space is undocumented, varies between studies, and is itself a confound: a transfer result obtained this way cannot be separated from the harmonisation that produced it.

We use the NetFlow v2 feature set of Sarhan et al. [5], built for this purpose. Its four datasets are released with an identical schema of 43 NetFlow features plus two label columns. The copies we use (below) omit the two IPv4 address columns, leaving 43 columns including the labels. We verify rather than assume that the schemas agree: the loader asserts that the numeric feature sets of all four are equal, not merely overlapping, and fails otherwise. We drop the two label columns and `L4_SRC_PORT` and `L4_DST_PORT`, leaving **39 features**; the loader would also drop the address columns if they were present. Ports are dropped because in these testbeds a destination port frequently identifies the attack script rather than the service. That is the spurious-correlation failure Arp et al. [17] describe.

The conventional CICFlowMeter pairing (CIC-DDoS2019 CICIDS2017) is retained as a secondary study, so the primary conclusions can be checked in the feature space the literature actually uses. The same CIC pair also appears, with a different protocol and question, in a separate unpublished manuscript by the author; the experiments reported here were designed and run independently of it.

Provenance. All seven datasets were obtained as Parquet copies from public Kaggle mirrors uploaded by one third party (Kaggle user `dhoogla`), not from the originators' distribution points: NF-CSE-CIC-IDS2018-v2, NF-BoT-IoT-v2, NF-ToN-IoT-v2 and NF-UNSW-NB15-v2 (mirror version 2), CIC-DDoS2019 and CICIDS2017 (version 3), and the DistriNet CICIDS2017 release (version 5, the 2022 files of Liu et al. [21]). Each mirror is labelled CC BY-NC-SA 4.0. The copies are pre-cleaned. The uploader reports removing records with missing values and duplicate records, and the four NetFlow copies hold 9.3% to 22.5% fewer rows than the totals the University of Queensland publishes for the original releases (Table 1). The CIC-DDoS2019 copy was down-sampled by its uploader to correct the original's extreme attack-to-benign imbalance, so its base rate is not that of the original capture. Row counts, duplicate rates and base rates in this paper therefore describe these copies. We do not redistribute any data; the code downloads the same mirror versions, and the originals are available from the University of Queensland, the Canadian Institute for Cybersecurity and DistriNet.

3.2 Labels

The positive class is any DoS or DDoS flow; the negative class is benign. Attacks that are neither, such as exploits, fuzzers, port scans and brute-force, are not silently discarded: the policy is an explicit parameter and every run records how many rows it moved, because discarding them changes the base rate and the base rate drives precision.

Labels also map to attack families (volumetric, reflection/amplification, low-rate application-layer) so transfer can be reported per family. Family assignment resolves the longest matching pattern across all families rather than the first family that matches: `DoS slowloris` contains both `dos` and `slowloris`, and a first-match rule files a low-rate attack as a flood. Ties break on a declared precedence, which places CIC-DDoS2019's `DrDoS_*` labels in reflection/amplification. The complete label → class → family map for all seven datasets is released.

3.3 De-duplication, and why it changes the picture

Exact duplicates are removed **in the feature space the model actually sees**, after socket columns are dropped. Two flows differing only in source port are one example to a model that never observes the port, and letting such a pair straddle a train/test split is leakage.

Table 1: Datasets after preparation. "Rows in copy" counts the mirrored copy used (§3.1); "official rows" is the total the University of Queensland publishes for the original NetFlow v2 release. Duplicate rate is measured in the feature space the model sees.

dataset	official rows	rows in copy	after de-duplication	duplicate rate	DDoS/DoS flows	base rate
NF-CSE-CIC-IDS2018	18,893,708	17,129,715	8,981,062	47.6%	453,237	0.0505
NF-BoT-IoT	37,763,497	30,420,086	101,369	**99.7%**	53,005	0.5229
NF-ToN-IoT	16,940,496	13,135,881	2,477,944	81.1%	529,884	0.2138
NF-UNSW-NB15	2,390,275	1,986,745	236,999	88.1%	3,075	0.0130
CIC-DDoS2019	—	431,371	418,884	2.9%	324,197	0.7740
CICIDS2017	—	2,313,810	2,231,536	3.6%	321,706	0.1442
CICIDS2017 (DistriNet 2022)	—	1,787,358	1,747,255	2.2%	266,899	0.1528

Three of the four NetFlow datasets are more than 80% duplicate. We additionally count *contradictory groups*, identical feature vectors carrying opposite labels, at 1,940 in NF-CSE-CIC-IDS2018 and 858 in NF-UNSW-NB15. Those counts are taken over the full attack taxonomy. Under the DoS-only policy the primary analysis actually uses, both fall to zero: every contradictory group in these two datasets pairs a benign flow with a *non-DoS* attack, and those rows are removed before modelling. The ceiling is therefore real for a general intrusion detector on these captures and absent for the DoS detector studied here, which is verified rather than assumed in §4.9.

3.4 Non-finite and impossible values

The per-second rate features carry finite but physically impossible magnitudes: `SRC_TO_DST_SECOND_BYTES` reaches 8.9×10^{304} in NF-ToN-IoT and `DST_TO_SRC_SECOND_BYTES` reaches 2.3×10^{252}, produced by dividing byte counts by near-zero flow durations. This is the extraction artefact Engelen et al. [15] document. Such cells describe the extractor, not the traffic. They are set to missing and counted (213 in NF-CSE-CIC-IDS2018, 216 in NF-ToN-IoT) rather than allowed to become silent infinities under a numeric cast.

3.5 Training

The order of operations is the substance of the protocol, and it is what carries the leakage guarantee:

> de-duplicate → split → fit on **source-train only** → transform everything else

Exact duplicates are removed from a dataset before any partition is drawn, so no feature vector can appear on both sides of a split. Splits are then stratified 80/20 and seeded. The median imputer and the scaler are fitted on source-train alone and applied unchanged to the source evaluation split and to every target, and the decision threshold is chosen on the source evaluation split (§3.6). No target label, target feature statistic or target-derived threshold enters the primary transfer experiment. A self-check verifies the scaler property on every run: a scaler fitted on train maps train exactly into [0, 1] and lets some unseen rows fall outside it, which a scaler that had seen those rows could not do.

Target features are clipped to the range the scaler observed; the clipping is ablated in §4.9, not assumed. Imbalance is handled with balanced sample weights and never by resampling, because SMOTE applied before a split leaks synthetic neighbours across it. The label-budget experiment of §4.12 does deliberately draw a balanced labelled adaptation set, but that is a controlled supervision budget rather than a training-time resampling choice.

Models are LightGBM (primary), XGBoost, RandomForest and LogisticRegression at fixed hyperparameters, with no tuning against any target. Five seeds are used for the transfer matrix and three for the ablations and remedy experiments, which are correspondingly more expensive; the asymmetry is a cost decision, and every reported interval states the number of pairs it rests on.

A stratified cap of 50,000 rows per class bounds cost, and it applies to **both** the source and the target: external metrics are computed on a seeded stratified sample of the de-duplicated target, not on the target in full. The cap equalises the two classes wherever both have at least 50,000 rows; a target with fewer attack flows, such as NF-UNSW-NB15 with 3,075, stays imbalanced. Either way it changes the evaluated prevalence, which moves every prevalence-sensitive metric. This is why ROC-AUC, which is invariant to prevalence under a fixed score ordering, carries the paper's central claim, while macro-F1 and MCC are read alongside it rather than in place of it.

3.6 Choosing the operating threshold

The decision threshold is chosen on the source evaluation split, never on target data, and taken as the midpoint of the widest tied-optimal plateau, over candidates placed between consecutive score levels.

This is not a detail. In-domain AUC here routinely reaches 1.0000, so a wide interval of thresholds is exactly optimal. Selecting by `argmax` returns whichever endpoint the grid visited first, and that endpoint sits flush against the score distribution, maximally fragile under any shift. We adopted the plateau-midpoint rule after a first run of the transfer matrix, in which the naive tie-break visibly moved the threshold loss of one pair; that pre-fix run was overwritten and is not reported, so we disclose the change here rather than quote its numbers. The rule uses source data only.

3.7 Metrics

To quantify the gap itself rather than its endpoints, we resample both test sets jointly and report a **paired bootstrap** interval and p-value on the difference (1,000 resamples). Reporting one interval for the in-domain score and another for the external score says nothing about the interval of their difference.

To separate a representation that stopped working from an operating point that went stale, we decompose each gap into **ranking loss** (Δ ROC-AUC) and **threshold loss** (external macro-F1 at the target-optimal threshold minus at the source threshold). The two have different remedies, and aggregating them hides which failure occurred.

To ask whether a detector's probabilities remain usable, we report **ECE** (15 equal-width bins) and **Brier score** in both domains, because a detector deploys a threshold rather than a ranking.

To keep the base rate visible, we report **FPR at 95% TPR**, since balanced-test-set F1 flatters every detector in this literature.

Table 2: Arp et al. [17] pitfalls and how this protocol addresses each.

Pitfall	Addressed by
Sampling bias	Four independently collected testbeds; every ordered pair in both directions; per-dataset base rates reported, not normalised away
Label inaccuracy	Full label→class map released; contradictory groups counted; original and DistriNet-regenerated CICIDS2017 compared under one source model (§4.15)
Data snooping	De-duplicate before splitting; scaler, imputer and threshold fitted on source data only, verified by the pipeline self-check (§3.5)
Spurious correlations	Socket columns dropped; feature classes ablated; `DNS_QUERY_ID` examined directly rather than trusted as noise
Biased parameter selection	Fixed hyperparameters across all domains; no tuning against any target
Inappropriate baseline	Four model families including logistic regression; a one-feature set as minimal reference
Inappropriate performance measures	macro-F1, MCC, AUC, PR-AUC, ECE, Brier, FPR@95%TPR reported together
Base rate fallacy	Base rates span 1.3%–77.4% and are stated; FPR at 95% TPR reported in Table 5; the balanced evaluation cap and its effect on prevalence-sensitive metrics stated in §3.5
Lab-only evaluation	Acknowledged in §6; all four datasets are testbed captures, which is part of the finding
Inappropriate threat model	Scope stated as DoS/DDoS detection under network change, not adversarial evasion

3.8 Software and AI-assisted tooling

The pipeline is written in Python (scikit-learn, LightGBM, XGBoost, pandas; versions pinned in the repository). Its code was developed with the assistance of Claude Code (Anthropic), an AI coding agent, using Claude models including Claude Opus 5.5 [CONFIRM: the Claude models used in August 2026]. Under the author's direction the agent wrote and modified the experiment, summary and build scripts and ran them. The study design and the primary experiments were specified by the author; the target-only control of §4.12 was designed with the agent in response to a pre-submission audit and approved by the author [CONFIRM]. The author reviewed the code and checked every reported number against the stored result files. The results do not depend on the tool: a clean re-run of the full pipeline from an empty results directory reproduced every result file and figure.

4 Results

Sections 4.1 to 4.4 establish what happens: the detectors look solved at home and invert abroad, in every model family and on every metric the protocol collects. Section 4.5 sorts those failures into the two kinds that need different repairs. Sections 4.6 to 4.9 ask what causes the inversion and whether our own preprocessing manufactured it. Sections 4.10 to 4.12 ask what an operator can see coming and what actually repairs it. Sections 4.13 to 4.15 give the scope conditions: which attacks are lost, which dataset pairings hide the effect, and how much of the measurement survives switching to a regenerated release of the target.

4.1 In-domain, the problem looks solved

On its own source evaluation split every source reaches what this literature routinely reports. LightGBM averages 0.9817 macro-F1 across the four sources (0.9546–1.0000) at ROC-AUC 0.9973–1.0000 (per-source means). The other three model families agree within a few points. For LightGBM, seed-to-seed standard deviations reach at most 0.0023; across all four families the largest is 0.0087 (logistic regression on ToN-IoT). Nothing here suggests an unsolved problem. These in-domain scores are computed on the same source split that chooses the threshold (§3.6), so in-domain macro-F1 is slightly optimistic and the gap correspondingly slightly overstated; ROC-AUC does not depend on the threshold.

Two features of these numbers matter for what follows. First, they are obtained *after* de-duplication, so they are not inflated by the train/test contamination described in §3.3; the uncorrected figures would be higher still. Second, ROC-AUC reaches 1.0000 on NF-CSE-CIC-IDS2018 to four decimal places, meaning the classes are perfectly separable in the source domain. Perfect in-domain separability is what makes the threshold plateau of §3.6 wide, and it is the first indication that the source decision rule may be exploiting testbed-associated structure rather than attack behaviour alone.

4.2 Out-of-domain, the failure has a sign

Applying the identical fitted model to another testbed across all 12 ordered pairs and 5 seeds, LightGBM averages 0.3788 external macro-F1 (0.1935–0.5957). The mean paired gap is 0.6029 and the paired bootstrap rejects a zero gap in **60 of 60** pair-seeds at $p < 0.05$ (Figure 1).

The informative statistic is external ROC-AUC. It averages 0.5054, which is chance. A mean at chance could describe a detector that has merely lost its signal. That is not what happened:

- **108 of 240** (model, pair, seed) runs score below 0.5;
- **149 of 240** have negative MCC;
- NF-UNSW-NB15 $\rightarrow$ NF-CSE-CIC-IDS2018 reaches **0.0697**, averaged over its five seeds.

Figure 2 shows the matrix. The entire NF-UNSW-NB15 source row is inverted against every target. This is the paper's central claim: cross-network failure can involve systematic ranking inversion rather than simple attenuation of the learned relationship. It is not the only mode observed: inversion is common rather than universal, and the taxonomy in §4.5 separates it from the milder modes.

4.3 The inversion is not a property of one learner

The same 12 ordered pairs and 5 seeds were run for four model families, which lets the question be asked directly rather than assumed away.

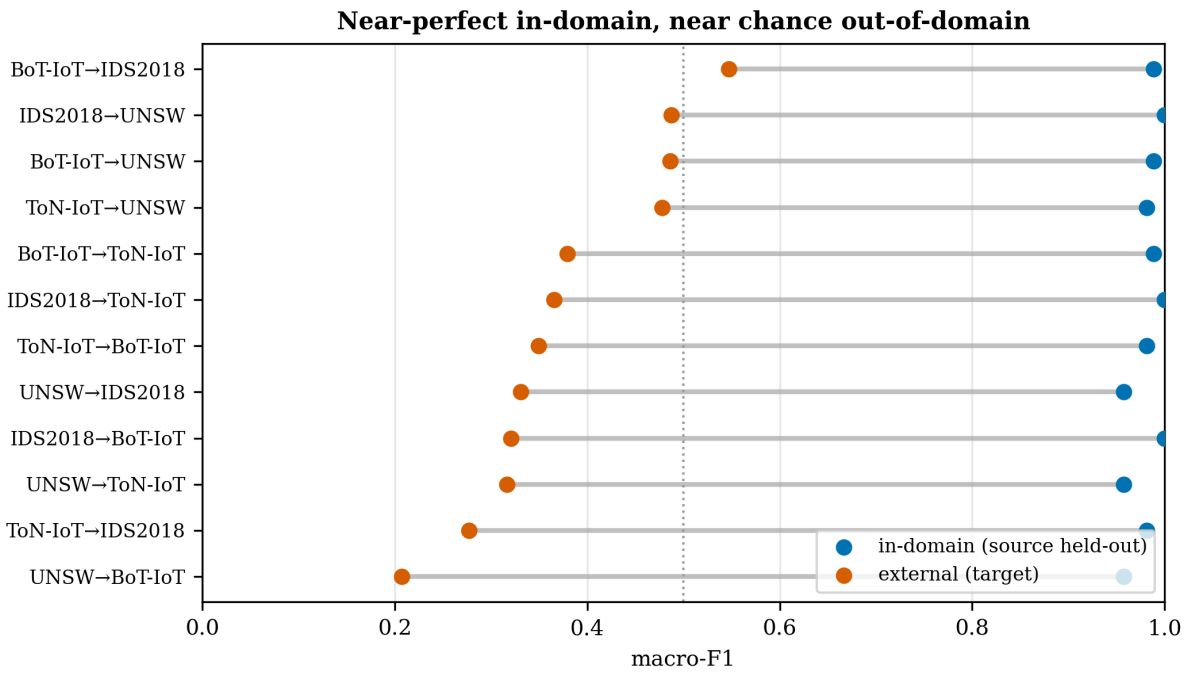


Figure 1: In-domain versus external macro-F1 for all 12 ordered pairs, LightGBM, mean over 5 seeds.

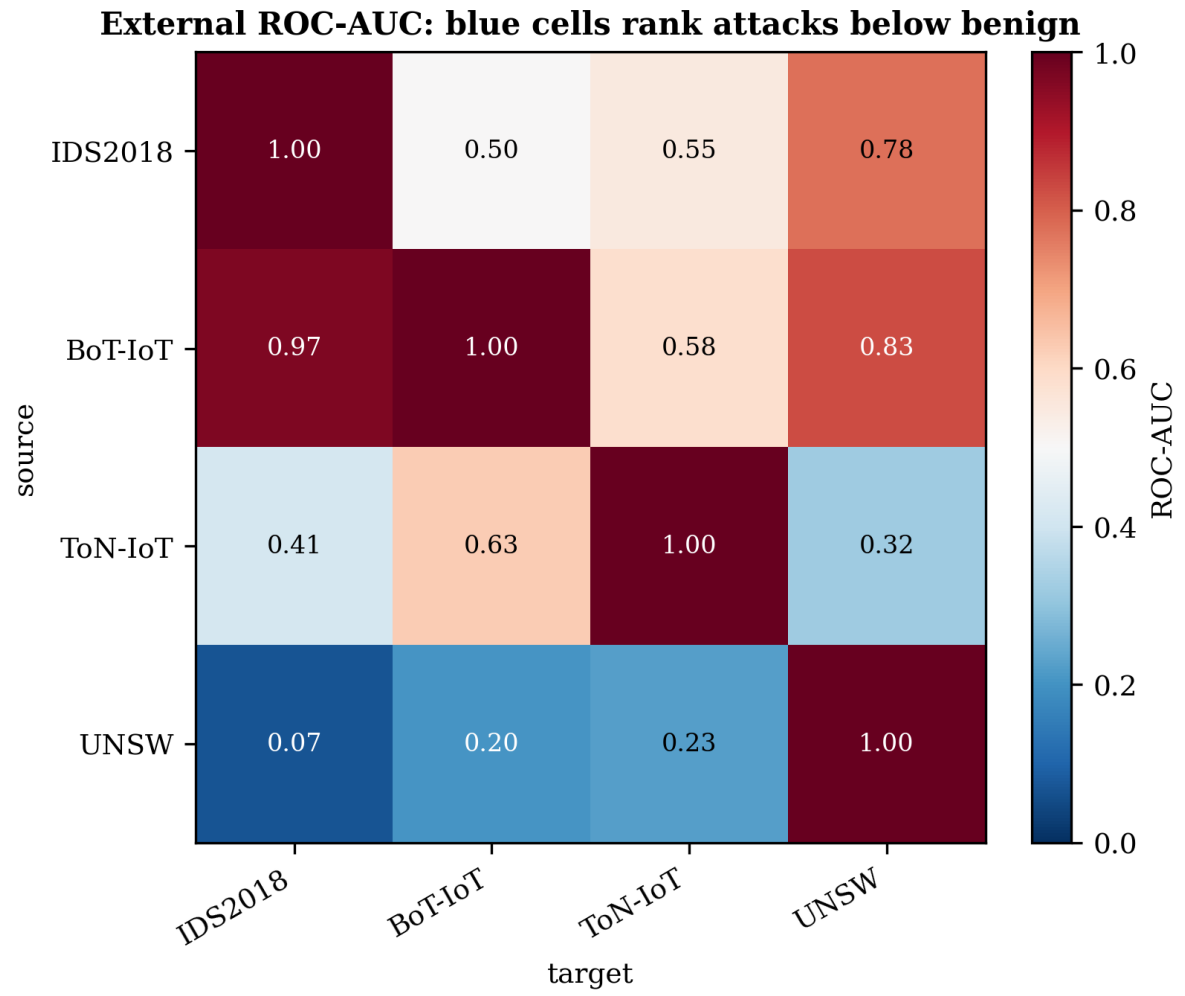


Figure 2: External ROC-AUC per ordered pair. Values below 0.5 indicate the ranking inverts in the target domain.

Table 3: Model-family robustness. External performance over all 12 ordered pairs and 5 seeds. Intervals are descriptive run-level bootstrap intervals over the 60 runs in each row; they are not pair-level inferential intervals, for the reason given below. The last column is the single worst run, not a pair mean.

model	n	mean external AUC	95% CI	below 0.5	mean macro-F1	mean MCC	lowest run
Logistic regression	60	0.5091	[0.4448, 0.5735]	41.7%	0.4789	+0.0590	0.0668
Random forest	60	0.5499	[0.4925, 0.6085]	40.0%	0.3665	−0.0596	0.1360
XGBoost	60	0.5184	[0.4537, 0.5840]	51.7%	0.3988	−0.0063	0.0339
LightGBM (primary)	60	0.5054	[0.4391, 0.5718]	46.7%	0.3788	−0.0459	0.0421

No family escapes. Mean external AUC lies between 0.5054 and 0.5499 for all four and the share of runs below chance between 40.0% and 51.7%; three of the four have negative mean MCC. Random forest transfers nominally best and still puts two runs in five below chance. Logistic regression carries the most weight here, because it is the one family that cannot represent interactions among features at all, and it inverts as readily as the ensembles do, with a worst run of 0.0668.

Averaging seeds within each pair shows that the failure is concentrated rather than diffuse.

Table 4: External ROC-AUC by ordered pair and model family. Mean over 5 seeds. Bold marks a ranking below chance.

source→target	LR	RF	XGB	LGBM	families below 0.5
BoT-IoT→IDS2018	0.9825	0.9742	0.9811	0.9656	0 of 4
BoT-IoT→ToN-IoT	0.6390	**0.4642**	**0.4569**	0.5833	2 of 4
BoT-IoT→UNSW-NB15	**0.3888**	0.7361	0.8191	0.8256	1 of 4
IDS2018→BoT-IoT	0.6717	0.6499	**0.4800**	0.5006	1 of 4
IDS2018→ToN-IoT	0.6884	**0.4967**	0.5805	0.5497	1 of 4
IDS2018→UNSW-NB15	0.6321	0.7791	0.7703	0.7750	0 of 4
ToN-IoT→BoT-IoT	0.6406	0.6064	0.7255	0.6279	0 of 4
ToN-IoT→IDS2018	0.6296	0.6848	**0.4904**	**0.4127**	2 of 4
ToN-IoT→UNSW-NB15	**0.0726**	0.5326	**0.3908**	**0.3241**	3 of 4
UNSW-NB15→BoT-IoT	**0.2334**	**0.2964**	**0.1910**	**0.2045**	4 of 4
UNSW-NB15→IDS2018	**0.3162**	**0.1729**	**0.0901**	**0.0697**	4 of 4
UNSW-NB15→ToN-IoT	**0.2139**	**0.2056**	**0.2449**	**0.2256**	4 of 4

Nine of twelve pairs fall below chance in at least one family; three do so in all four, and those three are exactly the transfers out of NF-UNSW-NB15, which §4.8 takes up separately. This is the more useful statement of the result. Inversion is not scattered noise that a different learner would average away: it attaches to particular source-target combinations and survives a change of hypothesis class, including a change to a model that can only add up single-feature contributions.

What counts as one observation. The counts above treat each (model, pair, seed) run as a unit, and those units are not independent — five seeds share a source dataset and four families share a split. Taking the ordered pair as the inferential unit instead, 5 of 12 pairs have a mean external AUC below chance, and the pair-level mean is 0.5054 [0.3589, 0.6529] over 12 pairs. That interval is wide, and it contains both substantial inversion and substantial above-chance transfer. We therefore do not claim that the population mean across conceivable network pairs is at or below chance; twelve pairs cannot support that. The claim this paper does defend is the one the interval leaves untouched: particular transfers invert systematically, repeatably across seeds, and in every model family tried. Throughout, run-level counts are reported as "of 240" and pair-level counts as "of 12", and the two are not interchangeable.

4.4 What the remaining metrics show

The rest of the protocol's metrics agree, and one of them is the operational headline. Table 2 commits to reporting PR-AUC, Brier score and FPR at 95% TPR alongside the ranking and threshold measures; they are collected on every run and summarised here.

Table 5: Probabilistic and operating-point metrics. Means over runs; 80 in-domain (4 sources x 4 models x 5 seeds) and 240 external. ECE is reported separately in §4.11 from a dedicated calibration run at a different sample size.

model	regime	PR-AUC	Brier	FPR@95%TPR
Logistic regression	in-domain	0.8727	0.0582	0.1127
Logistic regression	external	0.4886	0.3448	0.7519
Random forest	in-domain	0.9847	0.0095	0.0077
Random forest	external	0.4721	0.3448	0.7261
XGBoost	in-domain	0.9897	0.0092	0.0068
XGBoost	external	0.4721	0.4148	0.7063
LightGBM (primary)	in-domain	0.9900	0.0086	0.0066
LightGBM (primary)	external	0.4734	0.4294	0.7142

PR-AUC falls toward chance in every family, so the collapse is not an artefact of macro-F1's treatment of the two classes. Chance PR-AUC equals the evaluated prevalence: 0.5 where the cap balances the target, lower where it cannot. The Brier score rises by a factor between six and fifty. The number an operator would feel first is the last column: holding sensitivity at 95% of attacks, the false-positive rate on an unfamiliar network runs from 0.0066 to 0.7142 for the primary model. A detector that flagged about seven benign flows in a thousand at home flags seven in ten abroad, at the same recall.

4.5 Representation or operating point?

A collapsed macro-F1 admits two explanations with opposite remedies. The model may no longer separate the classes (a **ranking** failure, requiring retraining), or it may separate them well while the in-domain threshold has become meaningless (a **threshold** failure, requiring only recalibration).

We call a pair ranking-dominated when its ranking loss exceeds its threshold loss, where threshold loss is the macro-F1 regained by moving from the deployed threshold to the target-optimal one. Ranking loss can be measured two ways. As the drop in ROC-AUC, the comparison crosses units and is a convention; eleven of twelve pairs are ranking-dominated (Figure 3). As the macro-F1 still missing at the target-optimal threshold, both losses are in the same unit; nine of twelve are. Across the four model families the counts are 9 to 11 and 8 to 9 respectively (`results/decomposition_rule.json`), so the majority is robust and the exact count is not. The clearest exception, NF-BoT-IoT $\rightarrow$ NF-CSE-CIC-IDS2018, retains ROC-AUC 0.9656, a ranking that transferred nearly intact, while losing 0.4217 macro-F1 to a stale operating point.

The optimistic reading available in the literature, that cross-domain collapse is largely a calibration artefact, is therefore true here for one to three pairs in twelve. §4.11 measures what that is worth.

The decomposition also gives an operator a decision rule, though not a free one. Ranking loss requires target labels to compute, so it cannot be measured before deployment. What §4.10 shows is that a label-free proxy for the same quantity exists and is moderately informative, which is why the two sections are ordered as they are: the decomposition establishes that the distinction matters, and the prediction section asks whether it can be made without labels.

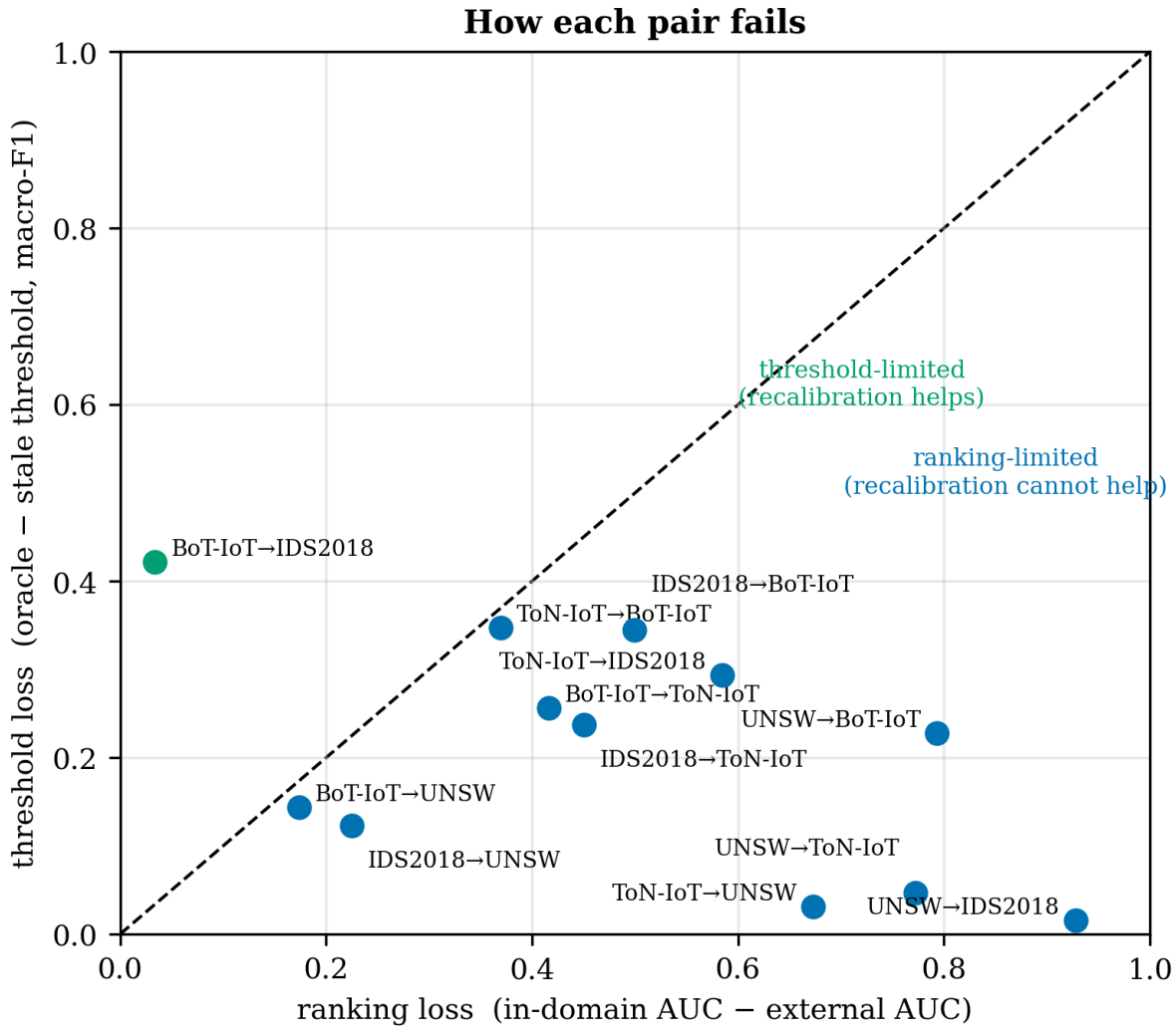


Figure 3: Ranking loss versus threshold loss per ordered pair. Points below the diagonal fail representationally; only one pair lies above it.

4.6 Which features cause it? A pre-specified hypothesis the data does not support

Flow features divide into those describing *how much traffic moved and how fast* (throughput, duration, byte and packet volumes) and those describing *what the endpoints did* (protocol, TCP flags, TTLs, retransmissions). The first group's values are set partly by the capturing testbed's link capacity and generator configuration. The hypothesis this study was designed around is that these **capacity features** inflate in-domain accuracy while carrying testbed-associated structure, so removing them should recover external performance. It was committed to the project repository before the experiment ran; the commit recording it precedes the commit carrying the result by two hours and seventeen minutes. The repository was private at that time and made public on release, so this is time-stamped pre-specification in version history rather than public pre-registration, and we claim only the former.

We assigned all 39 features to one of five semantic classes (20 capacity, 18 semantic, 1 held out) and trained under 17 feature sets. Each set is evaluated on the same (source, seed) combinations as the full-feature baseline, so differences are paired and the bootstrap resamples pairs.

Table 6: Capacity ablation, paired differences against the full 39-feature set.

feature set	k	external macro-F1	Δ vs `all`	95% CI	p	external AUC
`semantic_only`	18	0.3896	+0.0148	[−0.0077, +0.0341]	0.174	0.5394
`no_capacity`	19	0.3881	+0.0132	[−0.0084, +0.0320]	0.209	0.5386
`all`	**39**	**0.3748**	0	n/a	n/a	0.4931
`drop_rate_throughput`	35	0.3739	−0.0010	[−0.0033, +0.0013]	0.406	0.4882
`capacity_only`	20	0.3738	−0.0011	[−0.0293, +0.0314]	0.915	0.5221

Removing every capacity feature moves external macro-F1 by +0.0132, and the interval spans zero. The hypothesis is not supported. The last column adds something the pre-specified test could not see: on the ranking metric, capacity removal does move the needle, from 0.4931 to 0.5386, which is the difference between inverting and not. We flag this as an observation rather than a result — external AUC was not the pre-specified endpoint — but it points the same way as §4.7's finding that feature choices act on the operating point and the ranking separately. This is a failure to detect an effect, not a demonstration that none exists: the interval does not exclude effects up to roughly three points of macro-F1, and 12 source-seed pairs cannot resolve differences that size. Two caveats apply to every interval in Table 6. Seeds within a source share data, so the 12 units are closer to four independent ones and the intervals are narrower than they should be. And 16 feature sets are compared with `all`, uncorrected; at a Bonferroni level of 0.05/16, about 0.003, dropping the size and volume features ($p < 0.001$) survives and dropping the duration features ($p = 0.021$) does not.

The coarse grouping does, however, hide one real sub-effect. Dropping the 13 size and volume features alone yields +0.0615 [+0.0278, +0.0991], $p < 0.001$, and dropping the three duration features yields +0.0114 [+0.0009, +0.0277], $p = 0.021$. Byte and packet counts do carry testbed-associated structure in the way the hypothesis anticipated. What fails is the aggregation: bundling them with the throughput features, which contribute nothing (−0.0010 [−0.0033, +0.0013]), dilutes the effect to non-significance. A semantic partition chosen a priori was the wrong instrument, and we report that rather than re-cutting the classes until something reached significance.

4.7 Smaller sets transfer better, but not because they are smaller

Ordering the same 17 sets by external performance puts the smallest on top (Figure 4).

Every one of these differences is significant at $p < 0.001$. One feature out-generalises the entire standard set by 0.2763 macro-F1 [+0.2230, +0.3366].

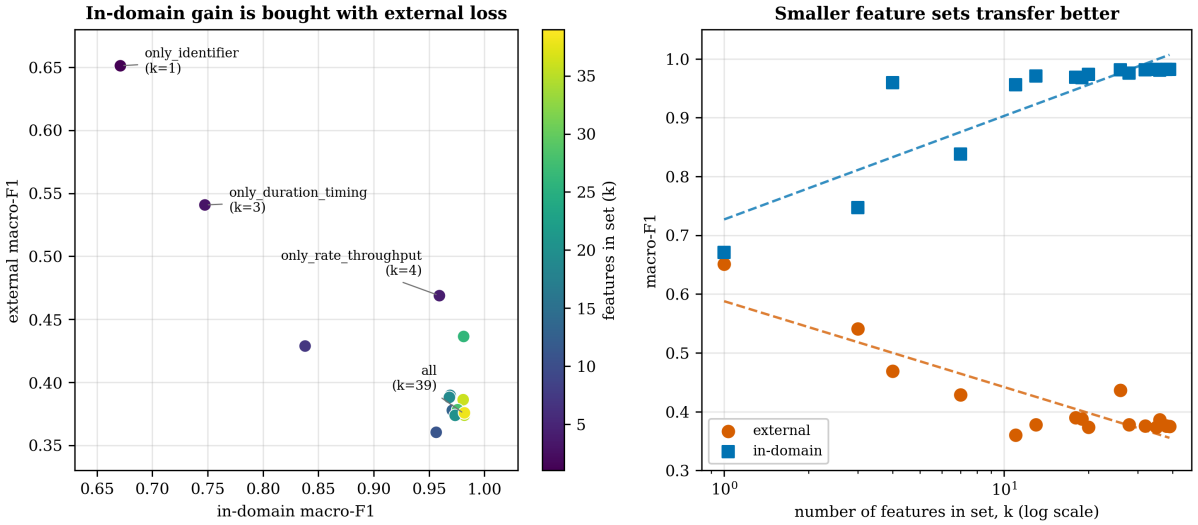


Figure 4: Feature-set size against in-domain and external macro-F1. The single-feature set attains the best external score of any subset tested.

Table 7: External performance by feature-set size.

feature set	k	in-domain	external macro-F1	Δ vs `all`	95% CI	external AUC
`only_identifier`	**1**	0.6711	**0.6512**	**+0.2763**	[+0.2230, +0.3366]	**0.7327**
`only_duration_timing`	3	0.7474	0.5407	+0.1659	[+0.0944, +0.2340]	0.5983
`only_rate_throughput`	4	0.9593	0.4688	+0.0939	[+0.0493, +0.1405]	0.6248
`only_flags_behaviour`	7	0.8380	0.4288	+0.0539	[+0.0371, +0.0705]	0.3759
`all`	39	0.9821	0.3748	0	n/a	0.4931

That ordering cannot, on its own, tell dimensionality apart from feature identity: a one-bit DNS indicator and a four-feature throughput block differ in both. We therefore ran a control that varies size alone. For each k in {1, 2, 3, 5, 7, 10, 15, 20, 30, 39} we drew 8 subsets of size k uniformly at random without replacement from the 39 features, and trained on each of the 4 sources under 2 seeds at 25,000 rows per class, with splits, model and targets held fixed. At k = 39 only one subset exists, so that cell contributes 1 rather than 8, giving 584 runs.

Table 8: Random subsets at controlled size. Median over subsets, sources and seeds; 5th and 95th percentiles in brackets.

k	in-domain	external macro-F1	external AUC
1	0.7158	0.5046 [0.2835, 0.6973]	0.5889
3	0.9250	0.4214 [0.2950, 0.5881]	0.5336
10	0.9774	0.4021 [0.2928, 0.5195]	0.5766
20	0.9842	0.3993 [0.2786, 0.5111]	0.5898
39	0.9843	0.3997 [0.3017, 0.4406]	0.5843

Size matters, and less than the semantic ablation implied. Across random subsets external macro-F1 falls from 0.5046 at k=1 to 0.3997 at k=39, Spearman -0.290 [-0.369, -0.208]. That accounts for **+0.105** of the one-feature advantage. The remaining **+0.147** is identity: `DNS_QUERY_ID` alone reaches 0.6512 where a random single feature reaches 0.5046. Both components are measured inside the control, against its own median 39-feature baseline of 0.3997, whereas Table 7's +0.2763 is a mean paired difference against the full-feature runs at 0.3748. One baseline is a median over random subsets and the other a mean over the full-feature set, so the two components do not sum to +0.2763 and are not a partition of it. What survives either normalisation is that dimensionality is the smaller share: most of the one-feature advantage is the feature, not the count.

The size component is a threshold effect, not a representational one. Feature count correlates strongly with in-domain macro-F1 (Spearman +0.626 [+0.565, +0.678]) and negatively with external macro-F1 (-0.290), but is uncorrelated with external ROC-AUC (+0.004 [-0.083, +0.091]). Adding features sharpens separation at home and leaves the target ordering unchanged; what degrades is the usefulness of the operating point chosen against that sharper separation. This connects directly to §4.5 and to §3.6: perfect in-domain separability widens the tied-optimal threshold plateau and pushes the chosen threshold into a position that does not survive a change of network.

An earlier draft explained the ordering by supposing that a gradient-boosted ensemble finds high-order conjunctions encoding its own testbed, which a one-feature model cannot express. The same two results that rule out a capacity account (§1) rule this out too: logistic regression inverts identically and cannot represent conjunctions, and feature count does not track external ranking. We record the discarded explanation because a reader who reaches the same intuition deserves to know it was tested.

A correction, stated where the claim depends on it. `DNS_QUERY_ID` was placed in its own class as a null control, on the assumption that a 16-bit DNS transaction ID is random and any accuracy from it would be memorisation by construction. That assumption is wrong.

The field is zero for every non-DNS flow, making it a one-bit "this flow is DNS" indicator; in three of four datasets DNS traffic is essentially never attack-labelled. The ID value itself carries next to nothing. In NF-CSE-CIC-IDS2018 and NF-BoT-IoT no attack flow has a nonzero ID, so AUC on the raw value equals AUC on the zero/nonzero bit; in NF-ToN-IoT and NF-UNSW-NB15 the two differ by at most 0.005 (0.4482 against 0.4529, and 0.1874 against 0.1879), and among nonzero IDs alone the raw value reaches AUC 0.48–0.49. The correct statement is therefore that a **one-bit DNS indicator out-generalises the 39-feature standard set**, and we read that as a finding about the benchmark rather than as a detector worth deploying. A field that separates the classes this well across three of four independently built testbeds, while carrying no information about attack behaviour, is a dataset-construction shortcut: DNS traffic was

Table 9: `DNS_QUERY_ID` is a protocol indicator, not an identifier. After the paper's preparation (DoS-only, de-duplicated); `results/dns_query_id.json`.

dataset	P(attack | id = 0)	P(attack | id ≠ 0)
NF-CSE-CIC-IDS2018	0.254	0.000
NF-BoT-IoT	0.785	0.000
NF-UNSW-NB15	0.051	0.001
NF-ToN-IoT	0.528	0.427

generated as background in these testbeds and was therefore almost never labelled as attack. Its in-domain score of 0.6711 is unusable in any case. What it establishes is that feature count and semantic quality are different things, and that the standard feature set contains at least one shortcut that transfers better than the behaviour the model is supposed to be learning.

4.8 Is the strongest inversion a weak-source artefact?

The three pairs that invert in every model family (§4.3) share a source, and that source retains only 3,075 attack flows after de-duplication at a 1.30% base rate. Source identity and source size are therefore confounded in the study's most extreme result, and the obvious alternative explanation is that a detector trained on very few positives simply fails unpredictably.

We separate the two by holding size fixed and varying identity: every source is starved to the same 3,075 attack flows that NF-UNSW-NB15 actually has, and the transfer matrix is rerun over two model families and three seeds.

Table 10: Source-size control. External ROC-AUC, 144 transfers.

regime	source	n	mean external AUC	below chance
matched (3,075 attacks)	NF-UNSW-NB15	18	0.3889	72.2%
matched (3,075 attacks)	other three sources	54	0.6155	25.9%
full cap	NF-UNSW-NB15	18	0.2513	88.9%
full cap	other three sources	54	0.6147	20.4%

Starving the other three sources does not make them invert. Their mean external AUC moves from 0.6147 at the full cap to 0.6155 when reduced to NF-UNSW-NB15's attack count, a difference of 0.0008. The inversion is therefore not explained by the number of positive training flows alone.

NF-UNSW-NB15 inverts under both regimes and remains 0.227 below the other sources at identical size. What separates it from the other three is therefore not size, and this experiment does not say what it is.

One secondary effect is worth stating because it is not what we expected. NF-UNSW-NB15's external AUC is *higher* under the matched regime (0.3889) than under the full cap (0.2513). The two regimes differ for this source in class balance rather than attack count: capping at 3,075 balances it, whereas the ordinary cap leaves roughly eight benign flows per attack. A more balanced source therefore inverts less severely, which is consistent with the threshold reading in §4.7 but is a single observation and we do not build on it.

4.9 Do the preprocessing choices manufacture the result?

Three decisions in §3.3 and §3.4 could in principle produce the effect this paper reports rather than expose it. Each is ablated over four sources and two seeds, holding everything else fixed.

Table 11: Preprocessing sensitivity. Mean over four sources and two seeds. The primary configuration is marked.

configuration	in-domain macro-F1	external macro-F1	external ROC-AUC
de-duplicated (primary)	0.9857	0.3952	0.5229
duplicates retained	0.9968	0.4345	0.5476
contradictory groups excluded	0.9857	0.3952	0.5229
clipped, LightGBM (primary)	0.9857	0.3951	0.5245
unclipped, LightGBM	0.9857	0.3951	0.5245
clipped, logistic regression (primary)	0.9195	0.4834	0.5295
unclipped, logistic regression	0.9195	0.5057	0.5460

De-duplication changes what the study would have concluded. Retaining duplicates lifts in-domain macro-F1 by 0.0111 and external macro-F1 by 0.0392, which makes the argument in §3.3 concrete: a study that splits these captures without removing exact duplicates reports in-domain numbers it has not earned.

The more consequential effect is on the external side, and it does not run in one direction. Retaining duplicates raises NF-UNSW-NB15's external ROC-AUC from 0.1872 to 0.4486, because its 1,915,838 raw rows collapse to 203,641 distinct ones and the surviving copies are not distributed evenly across the decision surface. Severe inversion would have been reported as ordinary near-chance failure. On NF-BoT-IoT the same choice moves external ROC-AUC the other way, from 0.7858 down to 0.6126. Duplicate contamination does not simply inflate results; it adds source-dependent noise to the very quantity this paper estimates, and on the strongest inversion in the study it would have hidden the finding.

Contradictory groups do not bind here. Excluding them changes nothing at all: the two rows of Table 11 are identical because, under the DoS-only policy, there are no such groups to exclude. Zero of 8,889,592 distinct feature vectors in NF-CSE-CIC-IDS2018 and zero of 203,641 in NF-UNSW-NB15 carry both labels. The 1,940 and 858 groups counted in §3.3 all pair a benign flow with a non-DoS attack and disappear with those rows. The arbitrary-survivor concern that motivates the check is therefore inapplicable to this analysis, which is worth stating precisely because the check was run expecting the opposite.

Clipping cannot matter for the tree ensembles, and slightly hurts the linear model. For LightGBM the ablation is exactly zero on every metric, and this is a property of the model rather than a measurement: every split threshold is learned inside the source training range, so clamping an out-of-range value to that range cannot move a row across any split. We report it because a null that follows from the model class is evidence about the design, not about the data, and reporting it as though it were the latter would be misleading.

Logistic regression is where the choice can bind, and there it is mildly harmful: not clipping improves external macro-F1 by 0.0223 and external ROC-AUC by 0.0165. Clipping touches between 4,014 and 51,468 target rows per source-seed, so the intervention is not negligible in scope. Neither manufactures the collapse: unclipped logistic regression still transfers at 0.5460 overall, and on NF-UNSW-NB15 it still inverts, at 0.4087.

None of the three decisions creates the effect. One of them, de-duplication, would have materially changed the conclusion had it been skipped, in the direction of understating what we found.

4.10 Can an operator predict the loss without labels?

An operator deploying a detector on their own network has traffic but no attack labels. We ask whether any label-free quantity predicts the loss, evaluated leave-one-pair-out, with all seeds of a held-out pair excluded together, against the baseline of predicting the mean gap (LOPO MAE 0.0827). A predictor that

cannot beat "assume the average drop" is not useful whatever its in-sample correlation.

Table 12: Label-free predictors of the observed gap.

predictor	inspects	Spearman	95% CI	LOPO MAE	vs baseline
ATC estimated accuracy	model outputs	**−0.726**	[−0.836, −0.532]	**0.0629**	**−24%**
mean confidence drop	model outputs	+0.503	[+0.168, +0.738]	0.0671	−19%
max per-feature Wasserstein	features	+0.433	[+0.103, +0.720]	0.0678	−18%
summed Wasserstein	features	+0.149	[−0.197, +0.491]	0.0930	+12%
proxy A-distance	features	−0.119	[−0.442, +0.242]	0.0849	+3%
RBF-MMD	features	+0.031	[−0.306, +0.385]	0.0910	+10%

The result splits by what the measure inspects (Figure 5). Measures of distance between feature distributions fail: proxy A-distance, RBF-MMD and summed Wasserstein all have intervals spanning zero and all three are worse than the mean baseline out-of-sample. The exception is the largest single-feature shift: the maximum per-feature Wasserstein distance reduces leave-one-pair-out error by 18% ($\rho = +0.433$ [+0.103, +0.720]). One feature moving far carries signal that the aggregate distance dilutes. With nine predictors on 36 observations and no multiplicity correction we treat it as a lead, not a result. This deserves stating plainly, because proxy A-distance in particular is the theoretical motivation behind a large body of domain-adaptation work. In this benchmark, these three marginal feature-distribution measures carry almost no information about how much a detector will lose. The result is not paradoxical: these measures describe the shift in $P(X)$, whereas the loss is driven by the shift in $P(Y \mid X)$, and the two need not move together. A benchmark on which they did would be a benchmark on which covariate shift dominated concept shift, and these four testbeds are evidently not that.

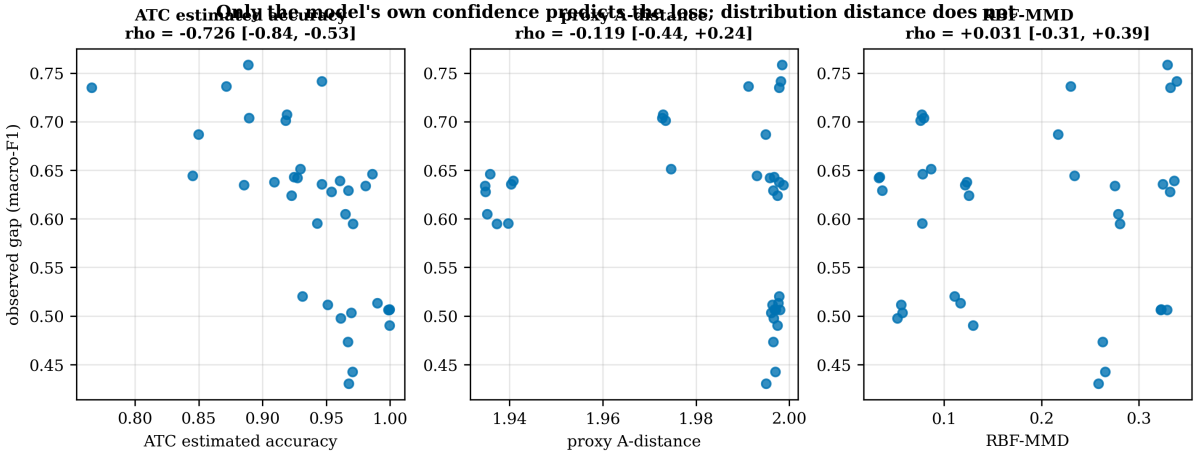


Figure 5: Observed generalization gap against three label-free predictors, with Spearman correlation and bootstrap CI.

Measures of the model's own behaviour work, moderately. ATC as defined by Garg et al. [19] picks the confidence threshold at which the source's own fraction of below-threshold points equals its error rate, then reports the fraction of target points above that threshold as a label-free estimate of target accuracy. We do not treat that estimate as an estimator of macro-F1; we use it as a *predictor*, fitting a univariate linear map from it to the observed external score and scoring that map leave-one-pair-out, as above. ATC reaches $\rho = -0.726$ against the gap and +0.756 [+0.585, +0.861] against external macro-F1, reducing leave-one-pair-out error by 24%. Twelve ordered pairs is a small sample, and a 24% error reduction supports triage, a decision about which deployments need labelled validation first, rather than a guarantee.

4.11 What recalibration and alignment can and cannot buy

Calibration collapses by two orders of magnitude. Mean ECE is 0.0043 in-domain and 0.4372 externally. These detectors do not become hesitant on an unfamiliar network; they stay confident and become wrong, which is the worse failure for an operator setting an alert threshold.

Benign-reference recalibration fixes the diagnosable minority, and harms others. A threshold read from the quantiles of the target's benign traffic requires no attack labels. We compute it on every target flow labelled benign, so it is an optimistic reference rather than a deployable zero-label method: it assumes the operator can isolate benign traffic perfectly, whereas a real sample of believed-benign traffic would contain some attacks and move the threshold. Its recovery, as a fraction of the headroom between the deployed threshold and the all-labels oracle, tracks external AUC almost exactly (Figure 6).

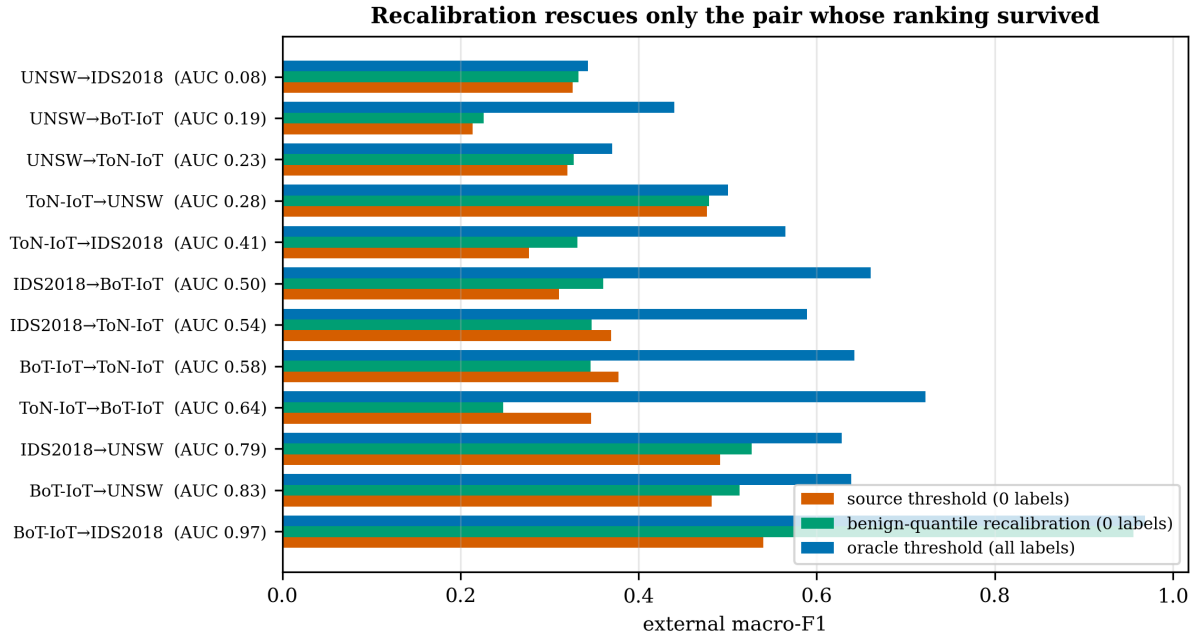


Figure 6: External macro-F1 under the deployed threshold, benign-reference recalibration (clean target benign traffic), and the all-labels oracle, ordered by external AUC.

It closes 97.0% of the one ranking-preserving pair and is actively harmful on three.

Pooled, the verdict depends on the summary. Scored on the held-out evaluation sets of §4.12, benign-reference recalibration reaches median macro-F1 0.3482 against 0.3919 for the deployed source threshold, but mean 0.4215 against 0.3953: it gains a great deal on a few pairs and loses a little on many. Neither pooled number is the useful reading. The per-pair table is: recalibration is worth doing when the ranking survived, and worth avoiding otherwise, which means an operator needs the diagnosis of §4.5 before choosing it rather than after. CORAL, a standard zero-label covariance alignment, recovers 0.0029 macro-F1 averaged over all pairs, which is indistinguishable from doing nothing. That figure comes from the first remedies run (`results/label_budget.json`, k = 0), whose label-using arms were superseded by the leakage-free design of §4.12; CORAL uses no target labels, so the leakage that invalidated those arms does not affect it. This bounds CORAL specifically; stronger representation-learning approaches to domain-invariant intrusion detection exist and are not evaluated here.

4.12 What ten labelled flows buy

Ten balanced labelled flows outperformed every baseline evaluated here that uses no attack labels. Each budget k is drawn as k/2 benign and k/2 attack flows from the adaptation pool. That isolates the value

Table 13: Benign-reference recalibration by pair. All 12 ordered pairs, LightGBM, mean over 3 seeds (`results/calibration.json`), sorted by headroom recovered. Headroom recovered = (benign-quantile − source threshold) / (oracle − source threshold); where the oracle barely exceeds the source threshold, a large percentage can be a small absolute gain.

pair	ext AUC	source thr	benign-quantile	oracle	headroom recovered
BoT-IoT→IDS2018	0.967	0.5399	**0.9558**	0.9688	**97.0%**
UNSW→IDS2018	0.079	0.3263	0.3325	0.3432	36.9%
IDS2018→UNSW	0.792	0.4918	0.5268	0.6280	25.7%
BoT-IoT→UNSW	0.829	0.4820	0.5138	0.6386	20.3%
ToN-IoT→IDS2018	0.414	0.2770	0.3311	0.5647	18.8%
IDS2018→BoT-IoT	0.501	0.3105	0.3605	0.6606	14.3%
UNSW→ToN-IoT	0.226	0.3204	0.3275	0.3704	14.1%
ToN-IoT→UNSW	0.284	0.4770	0.4792	0.5006	9.2%
UNSW→BoT-IoT	0.193	0.2135	0.2259	0.4403	5.5%
IDS2018→ToN-IoT	0.535	0.3690	0.3474	0.5890	−9.8%
BoT-IoT→ToN-IoT	0.583	0.3773	0.3465	0.6421	−11.7%
ToN-IoT→BoT-IoT	0.642	0.3469	0.2480	0.7220	**−26.4%**

of supervision from the difficulty of finding attacks — a controlled label-budget experiment, not a claim about live sampling.

Each target is split once per (pair, seed) into a disjoint adaptation pool and evaluation set. Labels are drawn only from the pool and every score is computed only on the evaluation set, including the baselines that use no attack labels, so the comparison is like for like. Draws are balanced, k/2 per class: at a 1.3% base rate a prevalence-proportional draw of ten flows usually contains no attack at all, which would measure the base rate rather than the value of supervision.

An earlier version of this experiment fine-tuned on k target rows and then scored a target set that still contained them, and tuned the threshold on rows it went on to score. We report the corrected design here. The direction of the error is worth stating: the contaminated run gave 0.7099 at k=10 against 0.7253 for the clean one, so the leak was depressing the estimate slightly rather than inflating it, and the conclusion is unchanged.

Ten balanced labelled flows lift median external macro-F1 from 0.3919 to 0.7253, a hundred reach 0.9116, and a thousand reach 0.9574 against an in-domain ceiling near 0.98 (Figure 7). Re-tuning the threshold on the same labels plateaus at roughly 0.61, and the reason is structural rather than empirical: changing a threshold cannot restore discrimination when the score ordering itself is reversed, because a threshold only chooses a cut point on an ordering it cannot change. Recovering the ordering requires changing the scoring function, which is what target labels buy.

The sharpest comparison is against a **target-optimal threshold oracle**: the best threshold obtainable on the evaluation set with unlimited target labels, which reaches 0.6158. It is an oracle rather than a zero-label method, since it consumes target labels without limit. Retraining on **ten** balanced labelled flows reaches 0.7253 and so exceeds it, and so does a classifier trained on the ten flows alone. Recalibration and supervision are not two points on one scale; beyond a certain point recalibration cannot buy what a few labels buy immediately.

The labels do the work, not the source model. A gain from retraining on source plus k target flows credits the source only if a model trained on the same k flows alone does worse. It does not. At k = 10, logistic regression trained on the ten flows alone reaches median macro-F1 0.7852 and a one-nearest-neighbour classifier 0.7704, both above the 0.7253 of source-plus-ten retraining; source-plus-ten beats the better of the two on only 27% of matched draws. The LightGBM configuration used elsewhere cannot

Table 14: Label budget, leakage-free. LightGBM. For k > 0, median over 240 runs per cell (12 ordered pairs × 2 seeds × 10 independent balanced label draws); for k = 0, over the 24 pair-seeds. 5th and 95th percentiles in brackets. This experiment caps each domain at 25,000 flows per class, half the 50,000 of the main matrix, to keep the refits tractable. The two target-only columns train on the same k labelled flows alone, matched draw for draw (`results/label_budget_target_only.json`).

k labelled target flows	re-tune threshold	retrain on source + k	target only: logistic regression	target only: 1-NN
0	0.3919 [0.2779, 0.5106]	0.3919 [0.2779, 0.5106]	—	—
10	0.5600 [0.2851, 0.9383]	**0.7253 [0.5604, 0.9540]**	0.7852 [0.6157, 0.9797]	0.7704 [0.6175, 0.9735]
50	0.6090 [0.3336, 0.9592]	0.8795 [0.7952, 0.9804]	0.8606 [0.7662, 0.9906]	0.8788 [0.7069, 0.9867]
100	0.6104 [0.3482, 0.9604]	0.9116 [0.8393, 0.9932]	0.9035 [0.8096, 0.9944]	0.9033 [0.7734, 0.9932]
500	0.6135 [0.3503, 0.9600]	0.9548 [0.8958, 0.9974]	0.9332 [0.8679, 0.9967]	0.9421 [0.8612, 0.9967]
1000	0.6142 [0.3620, 0.9609]	0.9574 [0.9228, 0.9986]	0.9425 [0.8831, 0.9969]	0.9494 [0.8899, 0.9973]

split ten rows and stays at 0.3333, so the comparison is made against learners suited to ten examples. From k = 50 the arms are within about two points of each other, source plus k slightly ahead of the two small-sample learners (Table 14), while the LightGBM configuration trained on the target alone overtakes source plus k from k = 100 (0.9335 against 0.9116). Per pair, the source helps at k = 10 on only 3 of 12 pairs, two of them among the three pairs whose external AUC exceeds 0.8; across all twelve the association between external AUC and the source's benefit is weak and uncertain (Spearman ρ = +0.34 [−0.41, +0.75], `results/target_only_per_pair.json`). We had read the fast recovery of the most inverted pairs as labels supplying the reversed sign of a still-informative ordering; the control does not support that reading, because the same pairs recover as well without the source.

The operational reading, at the scope the evidence supports, is that in this benchmark a small balanced labelled target sample was substantially more effective than the baselines tested that use no attack labels, and that what it buys is mostly a new target model rather than a repaired source model.

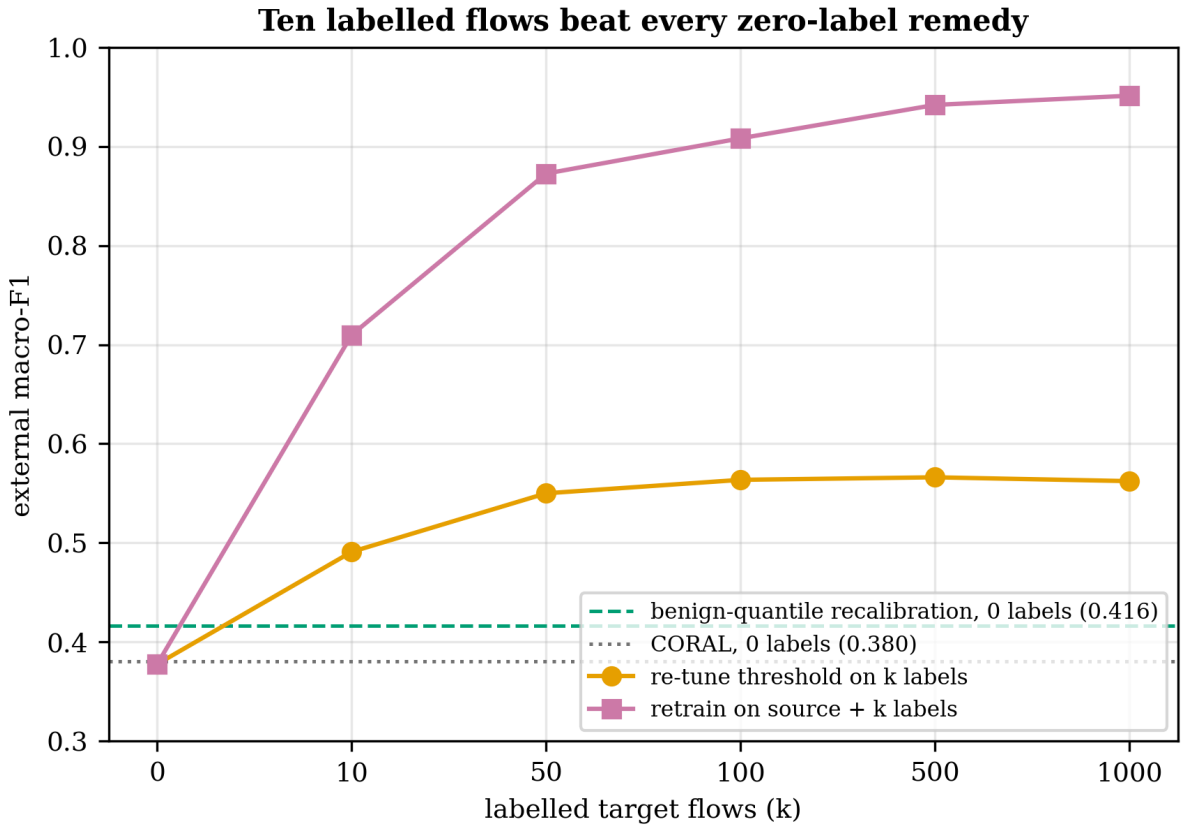


Figure 7: External macro-F1 against the number of labelled target flows, averaged over all ordered pairs and seeds.

4.13 Which attacks stop being detected

Table 15: Per-family recall under transfer. LightGBM; unweighted mean of per-run recall over the runs whose evaluation set contains the family. n = flows summed over those runs (in-domain / external).

attack family	in-domain	external	drop	n flows (runs)
volumetric	0.9834	**0.0535**	0.9299	153,784 (20) / 2,306,721 (60)
low-rate application-layer	1.0000	0.2358	0.7642	21 (5) / 324 (15)
benign (false-positive rate)	0.0118	0.0753	+0.0634	—

The in-domain low-rate recall of 1.0000 rests on 21 flows and should be read as an upper bound with wide uncertainty, not as perfect detection.

Volumetric floods, the largest and most conspicuous attacks and the class a detector is least expected to miss, transfer worst, retaining 5.35% recall. This follows directly from §4.2: a flood is defined by rates being high, and "high" is a property of the capturing testbed. Low-rate application-layer attacks, defined by request patterns rather than volume, had roughly four times the external recall, an estimate limited by how few low-rate flows survive de-duplication.

4.14 Dataset pairing changes the observed failure mode

Repeating the study on the conventional CIC-DDoS2019 CICIDS2017 pairing over 76 shared CICFlowMeter features reproduces the gap: in-domain 0.9675–0.9995, external 0.3368–0.4986, mean paired gap 0.5848, significant in 10 of 10 pair-seeds. Neither the gap nor its size depends on the NetFlow v2 choice.

The decomposition does differ. Where the NetFlow quartet is ranking-dominated in nine to eleven of 12 pairs (§4.5), the CIC pairing is **threshold**-dominated in most configurations, and RandomForest retains ROC-AUC 0.9348 across it with ranking loss 0.0651 against threshold loss 0.5643.

CIC-DDoS2019 and CICIDS2017 share an institution, a testbed lineage and an extractor. For the tree ensembles trained on either side, ranking largely survives that move, leaving an operating-point failure that recalibration can address. Not for every model: logistic regression trained on CIC-DDoS2019 inverts on CICIDS2017 (ROC-AUC 0.2417), and random forest in the same direction falls to 0.5638. Moving between independently constructed testbeds destroys the ranking instead. The failure mode therefore depends strongly on the dataset pair. We have one same-lineage pairing against a cross-lineage quartet, so this is evidence consistent with a lineage effect rather than a demonstration of one: CIC-DDoS2019 and CICIDS2017 may be unusually similar for reasons other than shared provenance. Under either reading, a study confined to one institution's dataset family risks overstating how recoverable the problem is.

4.15 A regenerated CICIDS2017 release makes transfer look worse

Table 16. Original versus DistriNet-regenerated CICIDS2017 as target, identical CIC-DDoS2019 source model. External macro-F1.

model	original	regenerated	Δ macro-F1
LightGBM	0.4827	0.3375	−0.1453
XGBoost	0.4986	0.3391	−0.1595
RandomForest	0.4824	0.3381	−0.1443
LogisticRegression	0.4543	0.3287	−0.1256

Mean $|\Delta|$ is 0.1437, consistent across all four model families. Replacing the original target with the 2022 regenerated release [21] lowers measured macro-F1 by 0.1256 to 0.1595, between 28% and 32% of the macro-F1 measured on the original. The regenerated release corrects labels but was also re-extracted with a repaired flow meter: it carries additional columns and 526,452 fewer rows. Labels and features therefore change together, and the comparison does not isolate label error. For LightGBM (seed 42) recall falls from 0.19 to 0.007 on the regenerated target, which a feature-distribution change explains at least as well as relabelling. What the result does establish is that the choice of release alone moves this cross-dataset number by close to a third, downward, so a cross-dataset result that uses the original CICIDS2017 as a target should be read as optimistic.

5 Discussion

Validation on one network provides no empirical guarantee of behaviour on another, and the failure it will exhibit is not conservatism. The detector keeps issuing confident verdicts while its ranking is wrong or reversed, at an external ECE two orders of magnitude above its in-domain value, and the aggregate feature-distribution measures evaluated here would not have raised the alarm, although the largest single-feature shift carried some signal. Monitoring the model's own confidence will, imperfectly.

This sharpens the prior measurements in §2.1 rather than contradicting them. Read as accuracy, those numbers say the signal is gone. Read as ranking, per ordered pair, the same collapse turns out to contain transfers that are not merely uninformative but systematically inverted. The distinction earns its keep operationally: it tells an operator that recalibration cannot help. It does not, on the evidence of §4.12, make the inverted source model worth repairing; ten target labels do as well without it.

The remedy hierarchy inverts the usual order of effort. Zero-label feature alignment by CORAL recovers 0.0029 macro-F1; a benign-reference threshold helps substantially in the one regime where ranking survived and hurts in three others; ten balanced labelled flows raise median macro-F1 from 0.3919 to 0.7253 with the source data and to 0.7852 without it. For an operator the implication is to build a small labelled set from the target network deliberately, covering both classes, rather than to rely on recalibration: the ten flows of §4.12 are drawn balanced, which a live sample of ten would not be.

The finding that a single protocol bit out-generalises a 39-feature standard set should not be read as an endorsement of one-feature detectors, since its in-domain score of 0.6711 is unusable. It is a statement about what the other 38 features are doing, which is encoding a testbed at a 0.2763 external cost.

What this does not settle. The experiments do not identify which feature-label associations change, or how; invariance across four hypothesis classes narrows the candidates without naming them. They do not show that unsupervised domain adaptation fails in general, only that one covariance-alignment method and one threshold heuristic did not recover the loss here. They do not show that drift monitoring is hopeless, only that three marginal divergence measures carried no usable signal across twelve pairs. And ten balanced labels are a controlled budget, not a deployment procedure. What the study does establish is a taxonomy of cross-network failure, and that which intervention helps depends on whether the target ranking survived.

Finally, §4.14 points to a methodological consequence beyond this paper. The one same-lineage pair tested here behaved differently from the cross-lineage quartet, consistent with a testbed-lineage effect on which failure mode appears — evidence for the pattern, not a demonstration of it. If it generalises, evaluating transfer only within one institution's dataset family, the field's common practice, understates the problem, which argues for cross-institution evaluation as the more informative default.

6 Threats to validity

All four datasets are testbed captures. None is production traffic, and the benign traffic of synthetic benchmarks differs statistically from real traffic [29]. The inversion could therefore be partly a product of how the testbeds encode attacks, and we cannot say whether production networks would invert. What the result does show is that a strong score on one research testbed is not evidence of transfer to another, which is the comparison most published cross-dataset numbers rest on.

The data are third-party copies. All seven datasets come from public Kaggle mirrors by one uploader (§3.1). The copies are pre-cleaned, hold 9.3% to 22.5% fewer NetFlow rows than the official releases, and in the case of CIC-DDoS2019 were down-sampled. Duplicate rates and base rates are therefore properties of these copies. We expect the direction of the main results to survive on the official files, because exact duplicates removed upstream would also have been removed by our own de-duplication, but we have not re-run on them.

Only classical supervised learners and one shallow adaptation baseline are tested. The four families are tree ensembles and logistic regression; neural detectors, unsupervised detectors, which generalise better on these NetFlow datasets [22] and on CIC captures [25], and adversarial domain adaptation [26], [27] are not evaluated. "Not a property of one learner" is a claim about these four families.

De-duplication is a consequential choice. Removing exact duplicates in the post-socket-drop feature space reduces NF-BoT-IoT by 99.7%. A reader who considers those rows independent observations would obtain higher in-domain scores and a different gap. We report the counts so either position can be evaluated; the alternative is untenable, since rows a model cannot distinguish cannot be independent test evidence for it.

The stratified cap alters base rates. Capping at 50,000 rows per class bounds cost but changes class balance, which moves precision-sensitive metrics. Base rates are reported throughout and operating-point metrics accompany F1 for this reason.

Non-DoS attacks are excluded from the primary analysis. Retaining them would change the base rate again; the policy is explicit and the counts moved are recorded, but the primary numbers describe a DoS/DDoS-versus-benign task.

Some "semantic" features may be testbed fingerprints. `MIN_TTL`, `MAX_TTL` and the TCP window sizes sit in the header/protocol class that §4.6 treats as describing endpoint behaviour. TTL and window values also encode the attacking hosts' operating systems and hop counts on each testbed, the kind of non-metadata contaminant D'hooge et al. [12] describe. A TTL-free ablation was not run, so the semantic classes of §4.6 should be read with that caveat.

Twelve ordered pairs is a small sample for the label-free prediction analysis. Leave-one-pair-out is the right protocol for the question, but a 24% error reduction estimated from twelve pairs supports triage, not a service-level guarantee. Resolving it needs more independently captured networks, not more seeds on these four.

NF-UNSW-NB15 is a small source. After de-duplication it retains 3,075 DoS flows at a 1.30% base rate, and it produces the study's most extreme inversion. The obvious confound is tested rather than conceded in §4.8: starving every other source to the same attack count leaves their transfer essentially unchanged (0.6147 to 0.6155), while NF-UNSW-NB15 remains 0.227 below them at identical size. Its class balance has a secondary influence that we note but do not interpret.

Low-rate attacks are underpowered in the NetFlow quartet. After de-duplication, NF-CSE-CIC-IDS2018 retains only 203 low-rate application-layer flows, so §4.13 rests partly on the CICFlowMeter secondary study. The per-family comparison there should be read as indicative, and the direction of any error is unknown rather than bounded.

Label error is unmeasured in the quartet. For CICIDS2017 we compare against the 2022 regenerated release, which changes features as well as labels. A regenerated CSE-CIC-IDS2018 also exists [21], but only in CICFlowMeter form; a NetFlow v2 version of it would have to be rebuilt from the pcaps, so residual label noise in the quartet is unmeasured. On the one target where a regenerated release can be applied, measured transfer moved down, so the quartet's uncorrected errors more plausibly flatter our external numbers than depress them, but this is an extrapolation from one target.

The mechanism behind the inversion is not isolated. Our experiments rule two explanations out without establishing a third — neither a gradient-boosting artefact nor model capacity expressed as feature count (§1, §4.7). What remains, and what we did not measure, is which individual feature-label associations change between networks, and how. Invariance across four hypothesis classes is consistent with a change in the low-order associations those classes have in common, but consistency is not identification, and we do not report it as a finding.

7 Conclusion

A flow-based DDoS detector moved to an independently constructed network does not become uncertain. In the cases studied here, it becomes confidently wrong. Across four testbeds, four model families and twelve ordered pairs, 108 of 240 external evaluations rank attack flows below benign ones, three ordered pairs do so in every model family tried, and the worst reaches ROC-AUC 0.0697: a ranking so consistently reversed that negating the score would yield 0.9303.

That distinction is the paper's practical content, because the two failures take different repairs. Where the ranking survives the move, a threshold read from target benign traffic recovers almost all of the loss without a single attack label. Where the ranking has reversed, no threshold restores the ordering at any budget, and ten balanced labelled target flows do more than an oracle handed unlimited labels to move the cut point, whether or not the source model is kept. The diagnosis has to come before the remedy, and §4.10 shows that a label-free proxy for it exists, imperfectly.

What any of this measures rests on how the data was prepared. These benchmarks are up to 99.7% duplicate in the feature space a model actually sees, and replacing one target with its regenerated release, which corrects labels and re-extracts features, moves measured transfer down by 0.1437 macro-F1. The next step the taxonomy points at, and the one this study stops short of, is identifying which individual feature-label associations change between networks, and how.

Data and code availability

Code, fixed seeds, configuration and result files are available at https://github.com/Rajveer-code/ddos-xdomain-paper. Every number in this paper is regenerated from `results/*.json` by the scripts that produced them. The datasets are not redistributed; they are available from the University of Queensland, the Canadian Institute for Cybersecurity and DistriNet, and the exact mirror versions used are listed in §3.1 and recorded in `results/provenance.json`.

References

1. D'Hooge, L., Wauters, T., Volckaert, B., and De Turck, F. "Inter-dataset generalization strength of supervised machine learning methods for intrusion detection." *Journal of Information Security and Applications* 54, 102564, 2020. doi:10.1016/j.jisa.2020.102564

2. Cantone, M., Marrocco, C., and Bria, A. "Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study." *IEEE Access* 12, 144489–144508, 2024. arXiv:2402.10974. doi:10.1109/ACCESS.2024.3472907

3. Hossain, M. Z., Khan, M. A. R., Islam, M. R., Islam, S. M. S., and Gedeon, T. "Assessing Generalisation Capability of Machine Learning Models for Intrusion Detection." *arXiv:2605.04407*, 2026.

4. Elangovan, R., Parthasarathy, D. D., Jawahar, M., Kaliyaperumal, P., Balusamy, B., Yogarayan, S., and Venkatesan, V. "Cross-Dataset Temporal and Semantic Generalization of Intrusion Detection Models for the Future Internet." *Future Internet* 18(4), 194, 2026. doi:10.3390/fi18040194

5. Sarhan, M., Layeghy, S., and Portmann, M. "Towards a Standard Feature Set for Network Intrusion Detection System Datasets." *Mobile Networks and Applications* 27, 357–370, 2022. arXiv:2101.11315. doi:10.1007/s11036-021-01843-0

6. Sharafaldin, I., Lashkari, A. H., and Ghorbani, A. A. "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization." *International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018. (CICIDS2017) doi:10.5220/0006639801080116

7. Koroniotis, N., Moustafa, N., Sitnikova, E., and Turnbull, B. "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset." *Future Generation Computer Systems* 100, 779–796, 2019. doi:10.1016/j.future.2019.05.041

8. Alsaedi, A., Moustafa, N., Tari, Z., Mahmood, A., and Anwar, A. "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems." *IEEE Access* 8, 165130–165150, 2020. doi:10.1109/ACCESS.2020.3022862

9. Moustafa, N., and Slay, J. "UNSW-NB15: a comprehensive data set for network intrusion detection systems." *Military Communications and Information Systems Conference (MilCIS)*, 2015. doi:10.1109/MilCIS.2015.7

10. Sharafaldin, I., Lashkari, A. H., Hakak, S., and Ghorbani, A. A. "Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy." *International Carnahan Conference on Security Technology (ICCST)*, 2019. (CIC-DDoS2019) doi:10.1109/CCST.2019.8888419

11. D'hooge, L., Verkerken, M., Volckaert, B., Wauters, T., and De Turck, F. "Establishing the Contaminating Effect of Metadata Feature Inclusion in Machine-Learned Network Intrusion Detection Models." *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA), Lecture Notes in Computer Science*, 23–41, 2022a. doi:10.1007/978-3-031-09484-2_2

12. D'hooge, L., Verkerken, M., Wauters, T., Volckaert, B., and De Turck, F. "Discovering Non-Metadata Contaminant Features in Intrusion Detection Datasets." *19th Annual International Conference on Privacy, Security and Trust (PST)*, 1–11, 2022b. doi:10.1109/PST55820.2022.9851974

13. D'hooge, L., Verkerken, M., Wauters, T., Volckaert, B., and De Turck, F. "Hierarchical feature block ranking for data-efficient intrusion detection modeling." *Computer Networks* 201, 108613, 2021. doi:10.1016/j.comnet.2021.108613

14. Goldschmidt, P., and Chudá, D. "Network intrusion datasets: A survey, limitations, and recommendations." *Computers & Security* 156, 104510, 2025. doi:10.1016/j.cose.2025.104510

15. Engelen, G., Rimmer, V., and Joosen, W. "Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study." *IEEE Security and Privacy Workshops (WTMC)*, 2021. doi:10.1109/SPW53761.2021.00009

16. Lanvin, M., Gimenez, P.-F., Han, Y., Majorczyk, F., Mé, L., and Totel, E. "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes." *Risks and Security of Internet and Systems (CRiSIS 2022)*, LNCS, pp. 18–33, 2023. doi:10.1007/978-3-031-31108-6_2

17. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., and Rieck, K. "Dos and Don'ts of Machine Learning in Computer Security." *31st USENIX Security Symposium*, 2022.

18. Axelsson, S. "The base-rate fallacy and the difficulty of intrusion detection." *ACM Transactions on Information and System Security* 3(3), 186–205, 2000. doi:10.1145/357830.357849

19. Garg, S., Balakrishnan, S., Lipton, Z. C., Neyshabur, B., and Sedghi, H. "Leveraging Unlabeled Data to Predict Out-of-Distribution Performance." *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2201.04234.

20. Sun, B., Feng, J., and Saenko, K. "Return of Frustratingly Easy Domain Adaptation." *AAAI Conference on Artificial Intelligence*, 2016. doi:10.1609/aaai.v30i1.10306

21. Liu, L., Engelen, G., Lynar, T., Essam, D., and Joosen, W. "Error Prevalence in NIDS datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018." In *Proc. IEEE Conference on Communications and Network Security (CNS)*, pp. 254–262, 2022. doi:10.1109/CNS56114.2022.9947235

22. Layeghy, S., and Portmann, M. "Explainable Cross-domain Evaluation of ML-based Network Intrusion Detection Systems." *Computers and Electrical Engineering* 108, 108692, 2023. doi:10.1016/j.compeleceng.2023.

23. Apruzzese, G., Pajola, L., and Conti, M. "The Cross-Evaluation of Machine Learning-Based Network Intrusion Detection Systems." *IEEE Transactions on Network and Service Management* 19(4), 5152–5169, 2022. doi:10.1109/TNSM.2022.3157344

24. Sarhan, M., Layeghy, S., and Portmann, M. "Evaluating Standard Feature Sets Towards Increased Generalisability and Explainability of ML-Based Network Intrusion Detection." *Big Data Research* 30, 100359, 2022. doi:10.1016/j.bdr.2022.100359

25. Verkerken, M., D'hooge, L., Wauters, T., Volckaert, B., and De Turck, F. "Towards Model Generalization for Intrusion Detection: Unsupervised Machine Learning Techniques." *Journal of Network and Systems Management* 30, 12, 2022. doi:10.1007/s10922-021-09615-7

26. Layeghy, S., Baktashmotlagh, M., and Portmann, M. "DI-NIDS: Domain invariant network intrusion detection system." *Knowledge-Based Systems* 273, 110626, 2023. doi:10.1016/j.knosys.2023.110626

27. Singla, A., Bertino, E., and Verma, D. "Preparing Network Intrusion Detection Deep Learning Models with Minimal Data Using Adversarial Domain Adaptation." In *Proc. 15th ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, pp. 127–140, 2020. doi:10.1145/3320269.3384718

28. Flood, R., Engelen, G., Aspinall, D., and Desmet, L. "Bad Design Smells in Benchmark NIDS Datasets." In *Proc. IEEE 9th European Symposium on Security and Privacy (EuroS&P)*, pp. 658–675, 2024. doi:10.1109/EuroSP60621.2024.00042

29. Layeghy, S., Gallagher, M., and Portmann, M. "Benchmarking the benchmark — Comparing synthetic and real-world Network IDS datasets." *Journal of Information Security and Applications* 80, 103689, 2024. doi:10.1016/j.jisa.2023.103689

30. Luay, M., Layeghy, S., Hosseininoorbin, S., Sarhan, M., Moustafa, N., and Portmann, M. "Temporal Analysis of NetFlow Datasets for Network Intrusion Detection Systems." arXiv:2503.04404, 2025.